

## Tugas Pemrograman Dasar - Fase 1

Nama: Rakha Danish Panular

Kelompok: Ruby

### Problem 1: The Last Astronaut

*Pseudocode:*

```
MULAI
  INPUT N, K
  BUAT array next[1..N]
  UNTUK i = 1 SAMPAI N-1
    next[i] = i + 1
  next[N] = 1

  start = 1
  prevStart = N
  remaining = N

  SELAMA remaining > 1
    predTarget = prevStart
    target = start
    UNTUK langkah = 1 SAMPAI K-1
      predTarget = target
      target = next[target]

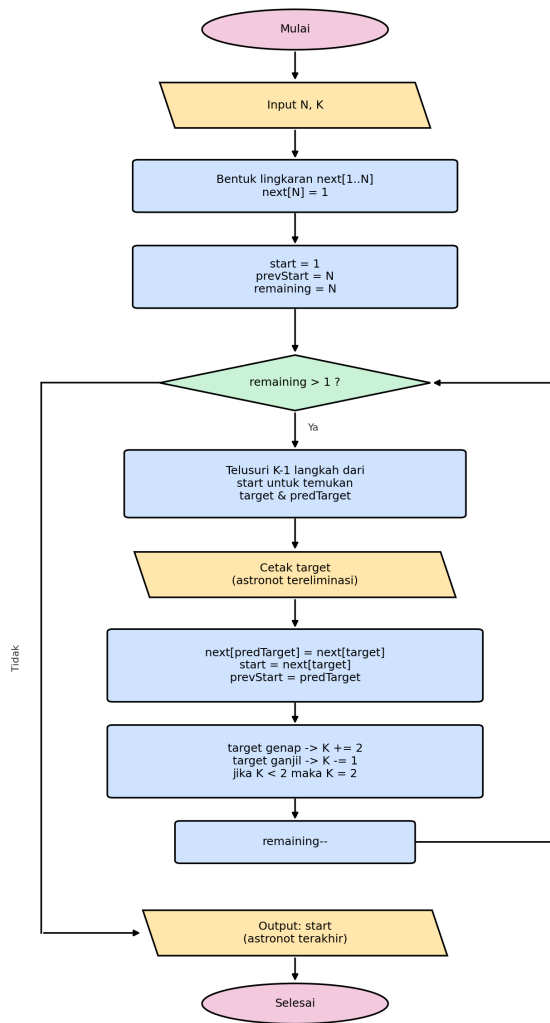
    CETAK target { astronot tereliminasi }

    next[predTarget] = next[target]
    start = next[target]
    prevStart = predTarget
    remaining = remaining - 1

  JIKA target GENAP MAKA
    K = K + 2
  LAINNYA
    K = K - 1
  JIKA K < 2 MAKA
    K = 2

  CETAK "Astronot terakhir yang bertahan:", start
SELESAI
```

*Flowchart:*



Jadi cara aku nyelesain soal ini tuh aku bayangin astronotnya berdiri melingkar, terus daripada aku simpen semua nomornya di array biasa terus digeser-geser tiap ada yang keeliminasi (ribet banget kalau gitu), aku pakai cara yang lebih gampang yaitu tiap astronot nyimpen "siapa yang ada di sebelahnya" lewat array next[]. Jadi next[i] itu isinya nomor astronot yang berdiri persis setelah astronot i. Nah pas ada yang keeliminasi, aku tinggal bikin astronot sebelum dia (predTarget) langsung nunjuk ke astronot setelah dia, otomatis yang keeliminasi kelewat/kelompatin gitu aja tanpa perlu geser-geser data.

Terus tiap putaran, aku jalan sejauh K-1 langkah dari titik mulai buat nemuin siapa yang bakal keeliminasi (soalnya titik mulainya sendiri udah dihitung sebagai hitungan pertama). Abis itu dicetak nomornya, terus titik mulai buat hitungan berikutnya pindah ke astronot yang tepat setelah yang barusan keeliminasi. Nilai K juga aku update tiap habis eliminasi, kalau nomor yang keeliminasi genap K nya nambah 2, kalau ganjil K nya berkurang 1, tapi kalau ternyata K nya jadi kurang dari 2 ya dipatok jadi 2 aja. Ini diulang terus pakai perulangan sampai astronot yang tersisa cuma 1, dan itulah astronot yang bertahan sampai akhir.

## Problem 2: Alien-in-the-Middle

*Pseudocode:*

```
FUNGSI hitungPanjang(pesan)
    panjang = 0
    SELAMA pesan[panjang] != karakter_kosong
        panjang = panjang + 1
    KEMBALIKAN panjang

FUNGSI sandikanPesan(pesan, hasil)
    panjang = hitungPanjang(pesan)
    nilaiSebelumnya = 0 { huruf pertama tidak punya huruf sebelumnya }

    UNTUK i = 0 SAMPAI panjang-1
        nilaiAsli = posisi_alfabet(pesan[i]) { A=1, B=2, ..., Z=26 }

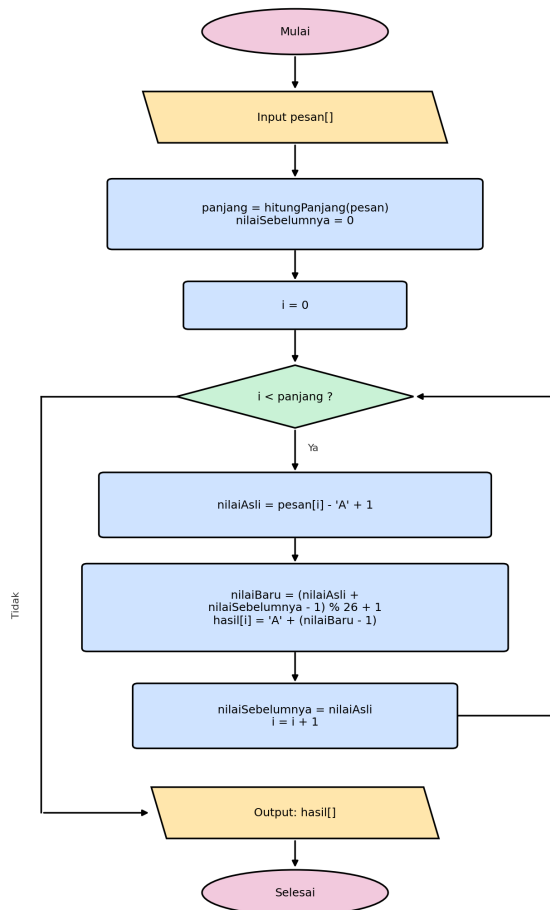
        nilaiBaru = ((nilaiAsli + nilaiSebelumnya - 1) MOD 26) + 1
        hasil[i] = huruf_dari_posisi(nilaiBaru)

        nilaiSebelumnya = nilaiAsli { simpan nilai huruf ASLI, bukan hasil sandi }

    KEMBALIKAN hasil

MULAI
    INPUT pesan
    hasil = sandikanPesan(pesan)
    CETAK hasil
SELESAI
```

*Flowchart:*



Kalau di soal ini, kuncinya adalah tiap huruf digeser berdasarkan huruf ASLI sebelum dia, bukan huruf yang udah disandi. Ini keliatan jelas dari contoh di soal, huruf ke-3 (I) itu digeser pakai nilai huruf ke-2 yang ASLI yaitu L (=12), bukan pakai huruf hasil sandi dari posisi ke-2 yang udah jadi M. Makanya di program aku bikin satu variabel nilaiSebelumnya yang isinya selalu nilai huruf asli dari iterasi sebelumnya, jadi nggak ketuker sama huruf yang udah disandi.

Huruf pertama nggak berubah sama sekali karena dia nggak punya huruf sebelumnya, makanya di awal nilaiSebelumnya aku set 0. Buat huruf-huruf selanjutnya, nilai barunya aku hitung dengan jumlahin nilai posisi huruf asli sama nilaiSebelumnya, terus di-mod 26 biar hasilnya tetap ada di rentang 1-26 (sesuai aturan soal kalau kelewat Z, muter lagi dari A). Rumus yang aku pakai itu  $((\text{nilaiAsli} + \text{nilaiSebelumnya} - 1) \text{ MOD } 26) + 1$ , soalnya posisi hurufnya mulai dari 1 bukan 0, jadi perlu disesuaikan dikit biar hasil muter-nya pas jatuh ke huruf yang bener. Ini diulang buat semua huruf di pesan sampai selesai semua, terus hasil akhirnya itu pesan yang udah tersandi.